Laboratory 5: Linear Models I

Ballotta Sofia, Bollengier Alexis, Di Ciano Paolo, Greco Giorgio
EE-512 Applied Biomedical Signal Processing
EPFL, Lausanne Switzerland 29/09/2025

Instructions

- This notebook provides all the questions of the practical session and the space to answer them. We recommend working directly here and then exporting the document as your report.
- Include any code used when addressing the questions, any figures you find useful, together with your answers.
- Please submit your report as a single PDF file, named according to: `name1_name2_name3_LabLinearModelsI.pdf`
- We recommend working in a group of 4–5 students (max 6); the report does not have to be submitted by every group member -- one submission per group is sufficient.

In [21]:
```
%matplotlib widget

# Imports
import os
import json
import numpy as np
import matplotlib.pyplot as plt

from statsmodels.tsa.ar_model import AutoReg, ar_select_order
from statsmodels.regression.linear_model import yule_walker

from scipy.signal import lfilter

# File paths
fbci = os.path.join(os.getcwd(), 'data', 'bci.json')
fafs = os.path.join(os.getcwd(), 'data', 'AF_sync.dat')
fspc = os.path.join(os.getcwd(), 'data', 'speech.dat')
fbld = os.path.join(os.getcwd(), 'data', 'blood.dat')
```

Experiment 1: classifying EEG signals

The file `/data/bci.mat` contains two data matrices, `left_hand` and `right_foot`, from a brain-computer interface (BCI) experiment. Each column in these matrices corresponds to a 2-second electroencephalography (EEG) recording (sampling frequency of 128 Hz) from the same electrode. The recordings in `left_hand` (respectively `right_foot`) were performed while the subject imagines a movement of the left hand (resp. right foot). The goal of the BCI experiment is to be able to "guess" what is being imagined based on the EEG signals alone.

We start by importing the signals (and removing their averages to better approximate our AR models):

In [22]:
```
with open(fbci, 'r') as f:
    jc = json.load(f)

eeg_lh = np.array(jc['left_hand'])
eeg_rf = np.array(jc['right_foot'])

eeg_lh -= np.mean(eeg_lh, axis=0, keepdims=True)
eeg_rf -= np.mean(eeg_rf, axis=0, keepdims=True)
```

AR model order

A useful first step to look for structure in the signals is estimating the AR model order of each signal. We define a function `ar_order` to fit models of different orders, obtain the model noise variance, and apply a specific criterion:

In [23]:
```
def ar_order(x, omax, Aff=0):
    """
    AR order estimation
    x: signal
    omax: maximum possible order
    Aff: 0 no graphic display; 1 display

    Returns:
    omdl: order estimated with MDL
    """

    nx = len(x)
    s = np.zeros((omax,))
    c = np.zeros((omax,))

    for k in range(omax):

        n = k+1

        ar_model = AutoReg(x, n, trend='n')
        ar_model_fit = ar_model.fit()
        sg2 = ar_model_fit.sigma2

        s[k] = sg2
        c[k] = nx * np.log(sg2) + (n+1) * np.log(nx)

    if Aff == 1:
        plt.figure()
        plt.plot(range(1, omax+1), mdl, 'o-')
        plt.title('Criterion')
        plt.show()

    return np.argmin(c)+1, s, c
```

Question 1.1. What is the name of the criterion being applied in the function `ar_order` implemented above? How would you change the code to apply the Akaike Information Criterion (AIC) instead?

Answer 1.1.

The criterion used is the Minimum Description Length (MDL):

$$MDL(p) = N \cdot ln(\sigma_p^2) + (p+1) \cdot ln(N)$$

Akaike information criterion (AIC) differs slightly from MDL:

$$AIC(p) = N \cdot ln(\sigma_p^2) + (p+1) \cdot 2$$

The code change affects just the following line:

`c[k] = nx * np.log(sg2) + (n+1) * np.log(nx)`

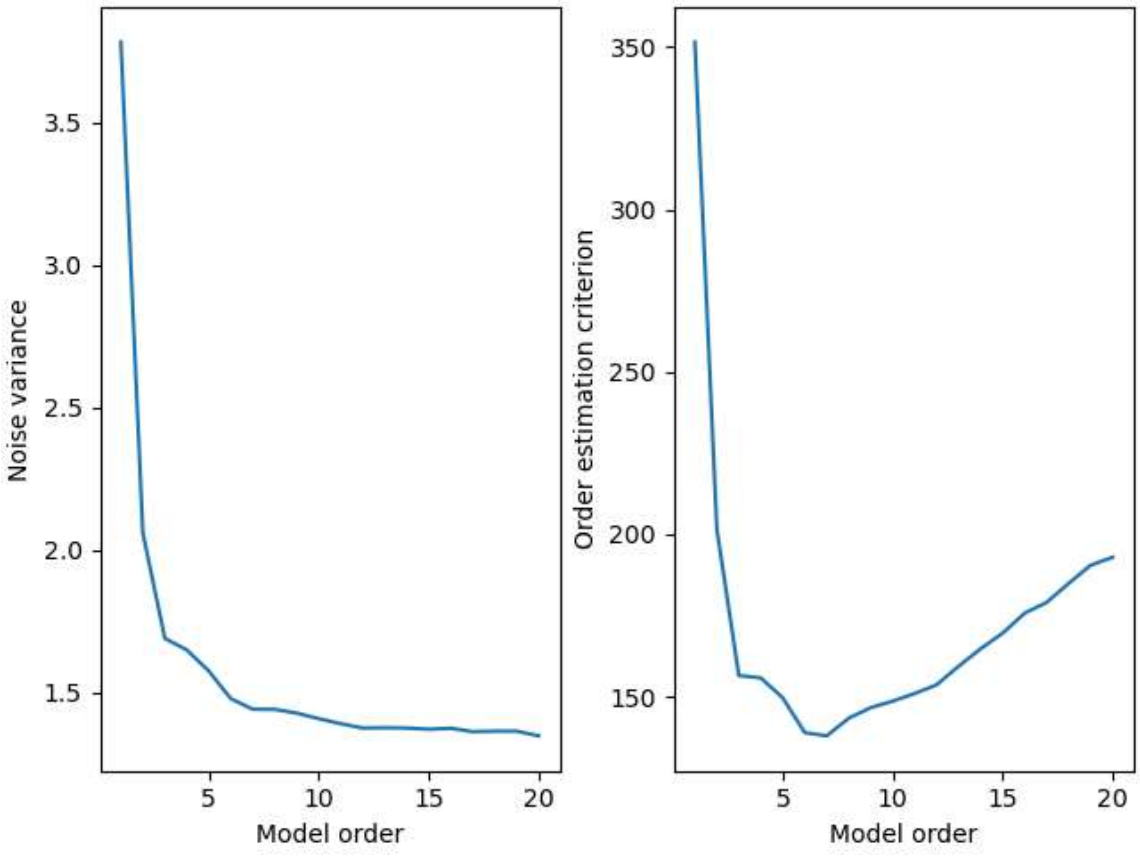
which could be changed to:

`c[k] = nx * np.log(sg2) + (n+1) * 2`

Let's look at the order estimate for an example signal, in a reasonable range up 20:

In [24]:
```
nmax = 20
_, s, c = ar_order(eeg_lh[:,1], nmax)

fig = plt.figure(constrained_layout=True)
n = np.arange(1, nmax+1)
plt.subplot(1,2,1)
plt.plot(n, s)
plt.xlabel('Model order')
plt.ylabel('Noise variance')
plt.subplot(1,2,2)
plt.plot(n, c)
plt.xlabel('Model order')
plt.ylabel('Order estimation criterion')
plt.show()
```

Figure



Question 1.2. Do the two curves obtained for this example signal behave as we would expect? What are their most important characteristics?

Answer 1.2.
Yes, they behave as expected because the **noise variance tends to decrease** as the model gets more complex.
However, the **order estimation criterion** (MDL in this case) **penalizes too complex models** (more specifically, it is linearly increasing with the model order and logarithmically increasing with the noise variance), even if they have very small noise variance. For this reason, the graph shows a U-shape behavior that identifies a minimum at model order 6 or 7

We can now estimate and print the optimal model order for every signal of each condition:

In [25]:
```
nmax = 10

print("\nLeft hand :", end="")
for k in range(eeg_lh.shape[1]):
    n, _, _ = ar_order(eeg_lh[:,k], nmax)
    print("  {:02.0f}".format(n), end="")
print("")

print("Right foot:", end="")
for k in range(eeg_rf.shape[1]):
    n, _, _ = ar_order(eeg_rf[:,k], nmax)
    print("  {:02.0f}".format(n), end="")
print("\n")
```

```
Left hand :  07  07  03  09
Right foot:  04  04  04  03
```

Question 1.3. Based on these AR order estimates, is there already a difference between the two categories overall? And if we wish to perform AR model estimation using a common choice of model order for all signals, which value should be chosen, and why?

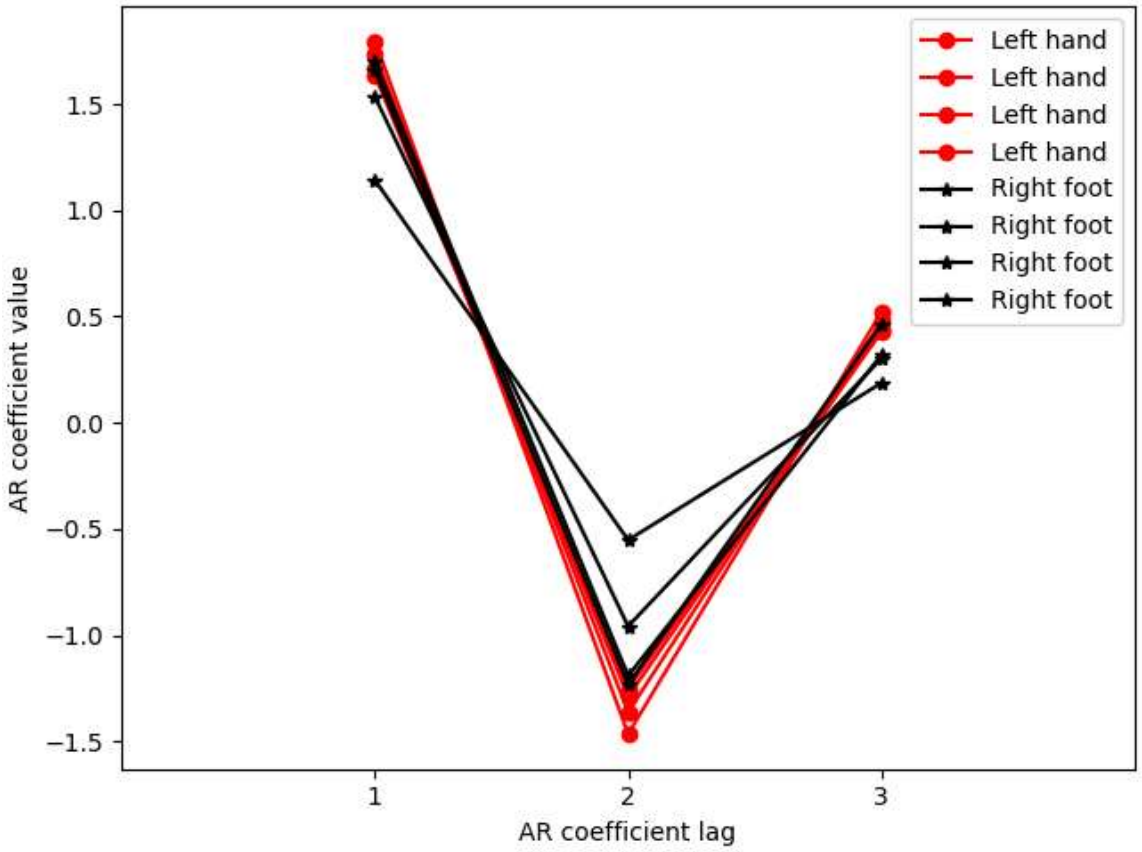
Answer 1.3.
The printed values show that the **left hand recordings** are well described by more complex models, suggesting **they express a more complex temporal structure** with respect to the right foot recordings. The third recording makes an exception to this reasoning, showing that even **a model of order 3 can well represent the data.**
If we had to choose the same model order to perform AR model estimation for both types of signal, it would be necessary to choose a high model order, between 6 or 9, which enables to either represent the complex behaviour of the left hand signals or the simpler right foot signals.

Movement prediction

We now look at the resulting AR coefficients when choosing a model order of `3`:

In [26]:
```
n = 3

fig = plt.figure(constrained_layout=True)

for k in range(eeg_lh.shape[1]):
    ar_model = AutoReg(eeg_lh[:,k], n, trend='n')
    ar_model_fit = ar_model.fit()
    plt.plot(range(1,4), ar_model_fit.params, 'ro-', label='Left hand')

for k in range(eeg_rf.shape[1]):
    ar_model = AutoReg(eeg_rf[:,k], n, trend='n')
    ar_model_fit = ar_model.fit()
    plt.plot(range(1,4), ar_model_fit.params, 'k*-', label='Right foot')

plt.legend()
plt.xticks(ticks=(1,2,3))
plt.xlabel('AR coefficient lag')
plt.ylabel('AR coefficient value')
plt.xlim(0,4)

plt.show()
```

Figure



Question 1.4. On which coefficients is the separation between categories most promising?

Answer 1.4.
Each point corresponds to the estimated AR coefficient value for a specific lag. The pattern of these coefficients characterizes the **temporal dynamics of the movement signal**. If two movement types have systematically different AR coefficients, these could serve as discriminative features for classification.

The **second coefficient shows the major differences** between the left hand measurements and the right hand ones.
Furthermore, the relation of a signal measurement with the one at 2 measurements before suggests also a rythmic or oscillatory behavior of the 2 signals categories. Further analysis (expecially spectral analysis) would be needed to confirm this hypothesis.

Experiment 2: AR model evolution over time

Real-life physiological signals can often vary substantially (and meaningfully) throughout a recording. It's usually good practice to have a look at the data before trying to apply models. Consider the signal in `AF_sync.dat` – a recording of ECG atrial activity during atrial fibrillation (sampling frequency of 50 Hz). We start by importing the signal:

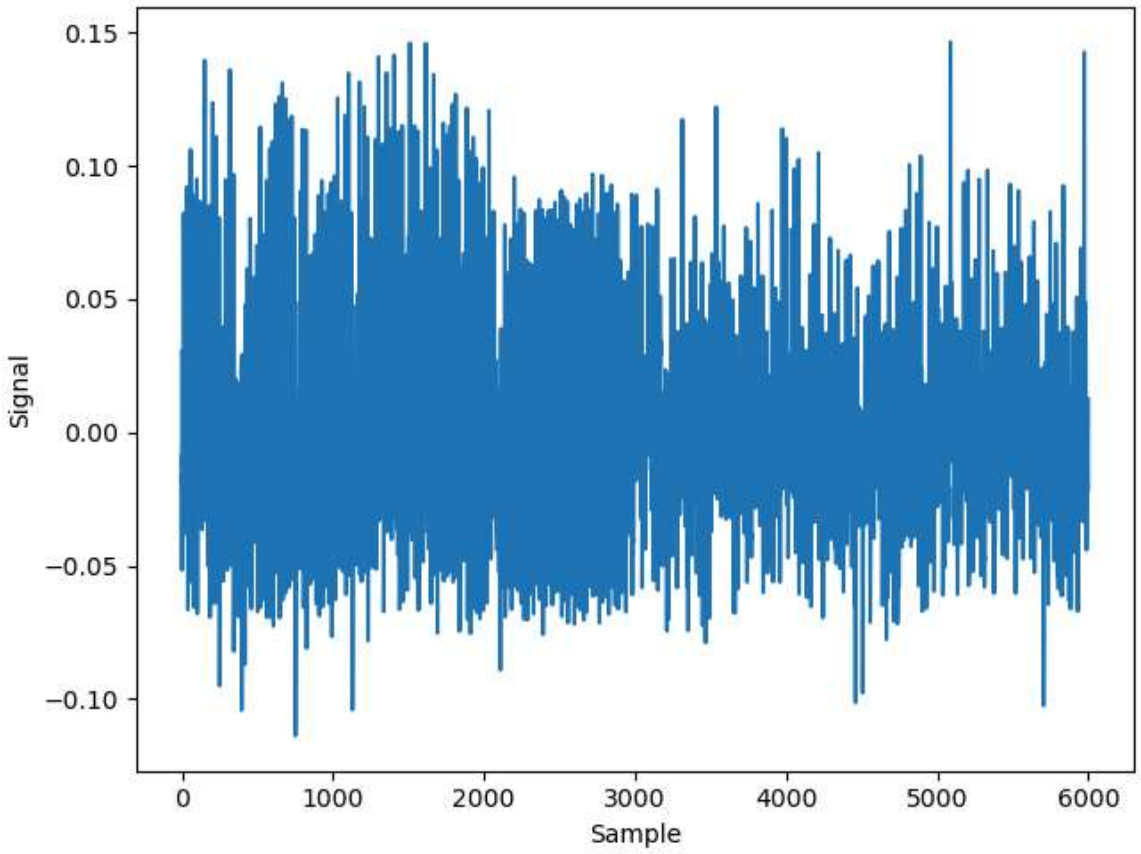
In [27]:
```
with open(fafs, 'r') as f:
    txt = f.readlines()
    af_sync = np.array([float(s[:-1]) for s in txt])

af_sync -= np.mean(af_sync)
```

Changes across time

Let's plot the signal and consider its evolution over the course of the recording. At the start (until sample ~2000 approximately), the signal is moderately organized; then it becomes very organized until sample ~3000. This probably corresponds to a drastic reduction in the number of fibrillatory waves in the atrial tissue (flutter). In the last part of the recording, the fibrillation, and thus the signal, becomes very disorganized.

In [28]:
```
fig = plt.figure(constrained_layout=True)
ax = plt.axes()
plt.plot(af_sync)
plt.xlabel('Sample')
plt.ylabel('Signal')
plt.show()
```

Figure

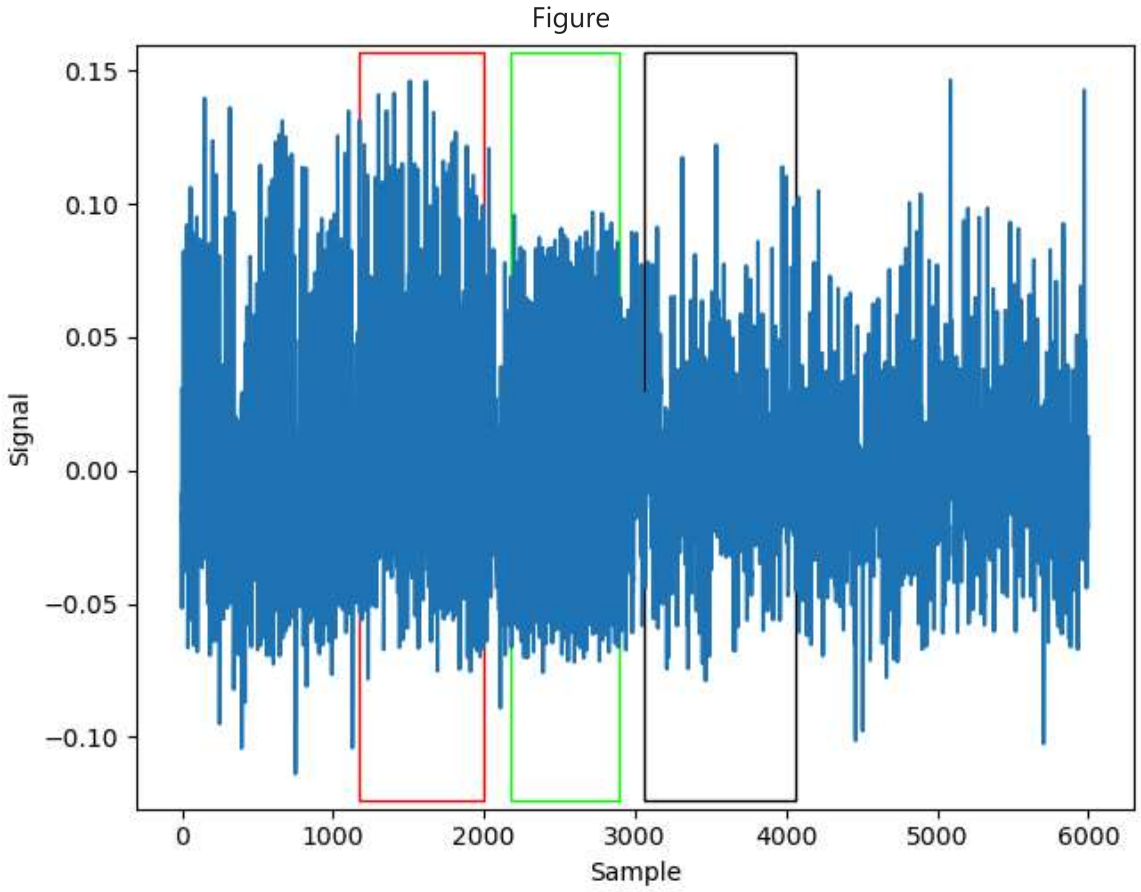


Question 2.1. The code below is intended to plot the signal again and mark the three periods described above, but it is not yet complete. Make the necessary modifications to show all three periods of interest.

Answer 2.1. The line of code that encontoure the period is: ax.axvspan(segments[0][0], segments[0][1], ymin=0.01, ymax=0.99, ec=eclr[0], fill=False)

So by just adding a loop it will highlight the different periods in segments

In [29]:
```
fig = plt.figure(constrained_layout=True)
ax = plt.axes()
plt.plot(af_sync)
plt.xlabel('Sample')
plt.ylabel('Signal')

segments = [[1170, 2000], [2170, 2890], [3060, 4060]]
eclr = [[1.0, 0.0, 0.0], [0.0, 1.0, 0.0], [0.0, 0.0, 0.0]]

for i in range(len(segments)):
    ax.axvspan(segments[i][0], segments[i][1], ymin=0.01, ymax=0.99, ec=eclr[i],

plt.show()
```



Adaptive modeling

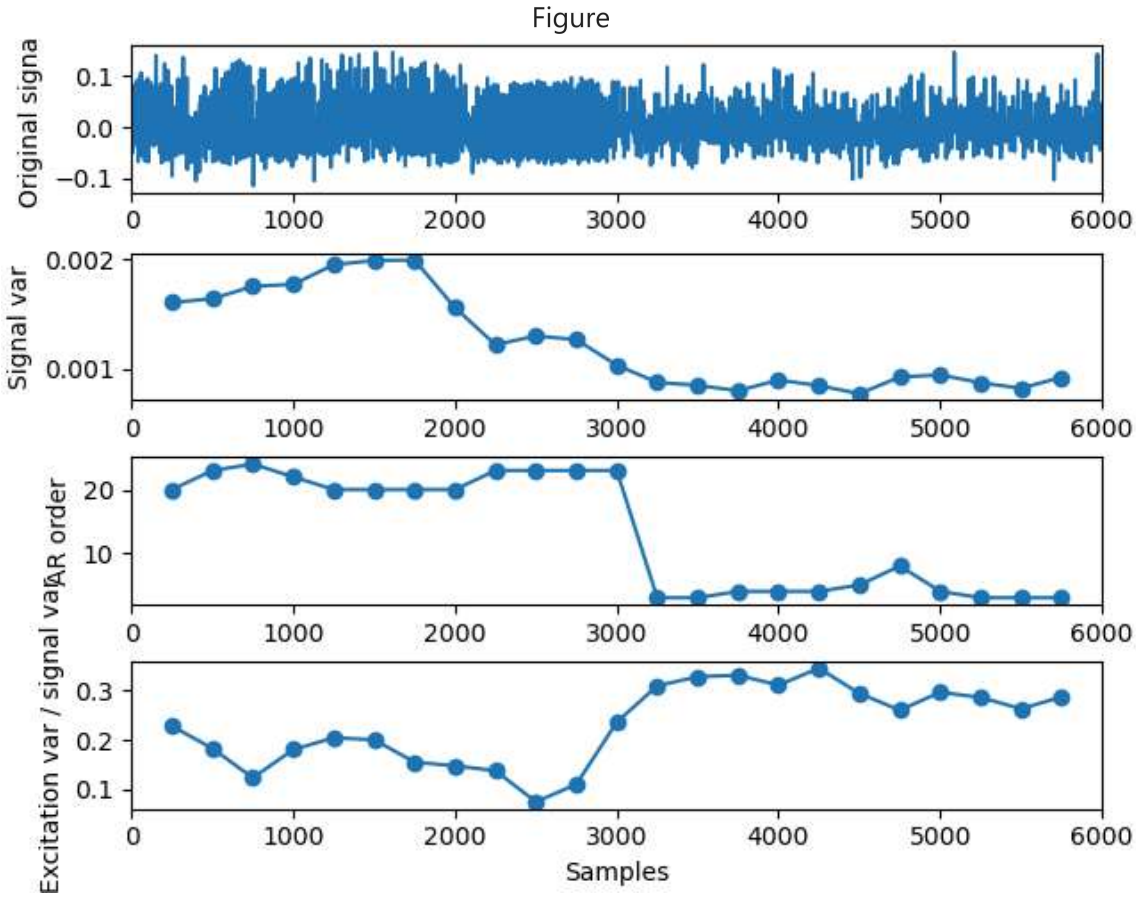
The clearly different states in `af_sync`, which vary across time, can be studied and modeled more quantitatively using a sliding-window approach. We thereby consider a segmentation of the signal into 500-sample windows with 50% overlap. For each segment, we can then estimate (i) the signal variance, (ii) optimal AR order, and (iii) the AR coefficients & excitation variance, as follows:

In [30]:
```
nw = 500
nv = round(nw*0.50)
nt = len(af_sync)

kt = []
ki, kf = 0, nw

# Getting the border and middle indices for each segment
while True:
    kt.append([ki, round(0.5*(ki+kf)), kf])
    ki += nw - nv
    kf += nw - nv
    if kf > nt: break

nc = len(kt)
arv = np.zeros((nc,))
aro = np.zeros((nc,), dtype=int)
arr = np.zeros((nc,))
nmax = 40

# Modeling the signal for each segment
for kc in range(nc):
```

```
    ys = af_sync[kt[kc][0]:kt[kc][2]]

    arv[kc] = np.var(ys)
    aro[kc], _, _ = ar_order(ys, nmax)

    _, sigma = yule_walker(ys, order=int(aro[kc]), method="mle")
    arr[kc] = sigma**2 / arv[kc]
```

We can then plot together the time evolution of the raw signal, the signal variance, the AR order, and the ratio of excitation variance to signal variance.

In [31]:
```
fig = plt.figure(constrained_layout=True)
t = np.array(kt)[:,1]

plt.subplot(4,1,1)
plt.plot(af_sync)
plt.ylabel('Original signal')
plt.xlim(0, len(af_sync))

plt.subplot(4,1,2)
plt.plot(t, arv, 'o-')
plt.ylabel('Signal var')
plt.xlim(0, len(af_sync))

plt.subplot(4,1,3)
plt.plot(t, aro, 'o-')
plt.ylabel('AR order')
plt.xlim(0, len(af_sync))

plt.subplot(4,1,4)
plt.plot(t, arr, 'o-')
plt.ylabel('Excitation var / signal var')
plt.xlim(0, len(af_sync))
plt.xlabel('Samples')

plt.show()
```



Question 2.2. Interpret the time evolution of the parameters plotted above, and how they relate to the organization of the signal in the three afore-mentioned stages.

Answer 2.2. `Write your answer here`

When the signal's activity is regular and periodic, it has a strong temporal structure and can be well modelled by a high-order autoregressive (AR) process with low excitation variance. Before around 3,000 samples (plots 1-2), the AR order is high (around 20-22) and the excitation variance is small, indicating an organised, correlated regime. Around 3,000 samples (plot 3), the AR order decreases to ~4-6 and the excitation-to-signal variance ratio increases (plot 4), indicating a transition to a less organised, more noise-driven phase. Afterwards, both stabilise at lower values, marking a simpler, less structured regime.

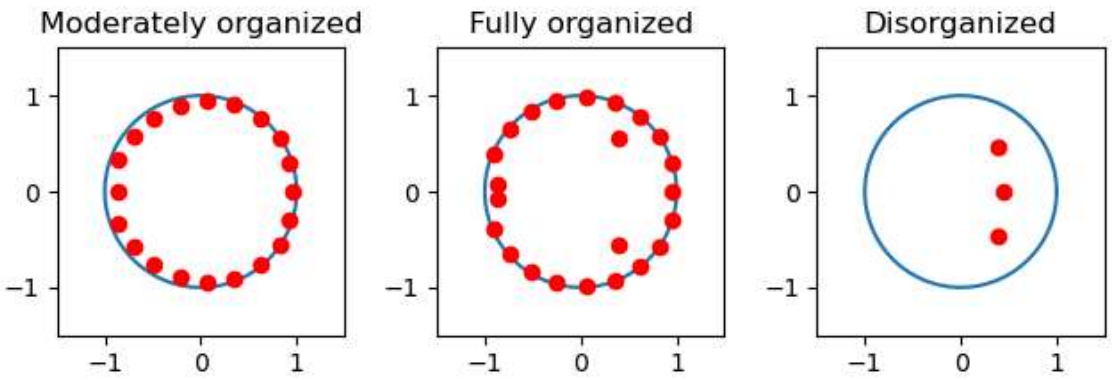
Signal stability and organization

Finally, another way of evaluating how organized a signal is, is by looking at its equivalent filter transfer function *H(z)* and the positioning of the corresponding poles. We can focus specifically on the three states defined previously in `segments`, as follows:

In [32]:
```
roots = []

for seg in segments:

    ys = af_sync[seg[0]:seg[1]]

    aro, _, _ = ar_order(ys, nmax)
```

```
    rho, _ = yule_walker(ys, order=int(aro), method="mle")

    rho = np.concatenate((-rho[::-1], np.array([1])), axis=0)
    roots.append(1 / np.roots(rho))

fig = plt.figure(constrained_layout=True)
t = np.linspace(0, 2*np.pi, 1000)
titles = ['Moderately organized', 'Fully organized', 'Disorganized']

for k in range(3):
    plt.subplot(1,3,k+1)
    plt.plot(np.cos(t), np.sin(t))
    plt.plot(np.real(roots[k]), np.imag(roots[k]), 'ro')
    plt.axis('square')
    plt.xlim([-1.5, 1.5])
    plt.ylim([-1.5, 1.5])
    plt.title(titles[k])

plt.show()
```

Figure



And the proximity to the unit circle can be further summarized in terms of the average magnitude:

In [33]:
```
for kr in range(len(roots)):
    print(titles[kr] + ': {:.3f}'.format(np.mean(np.abs(roots[kr]))))
```

```
Moderately organized: 0.943
Fully organized: 0.951
Disorganized: 0.550
```

Question 2.3. Comment on how signal organization relates to pole location (proximity to the circle).

Answer 2.3. `Write your answer here`

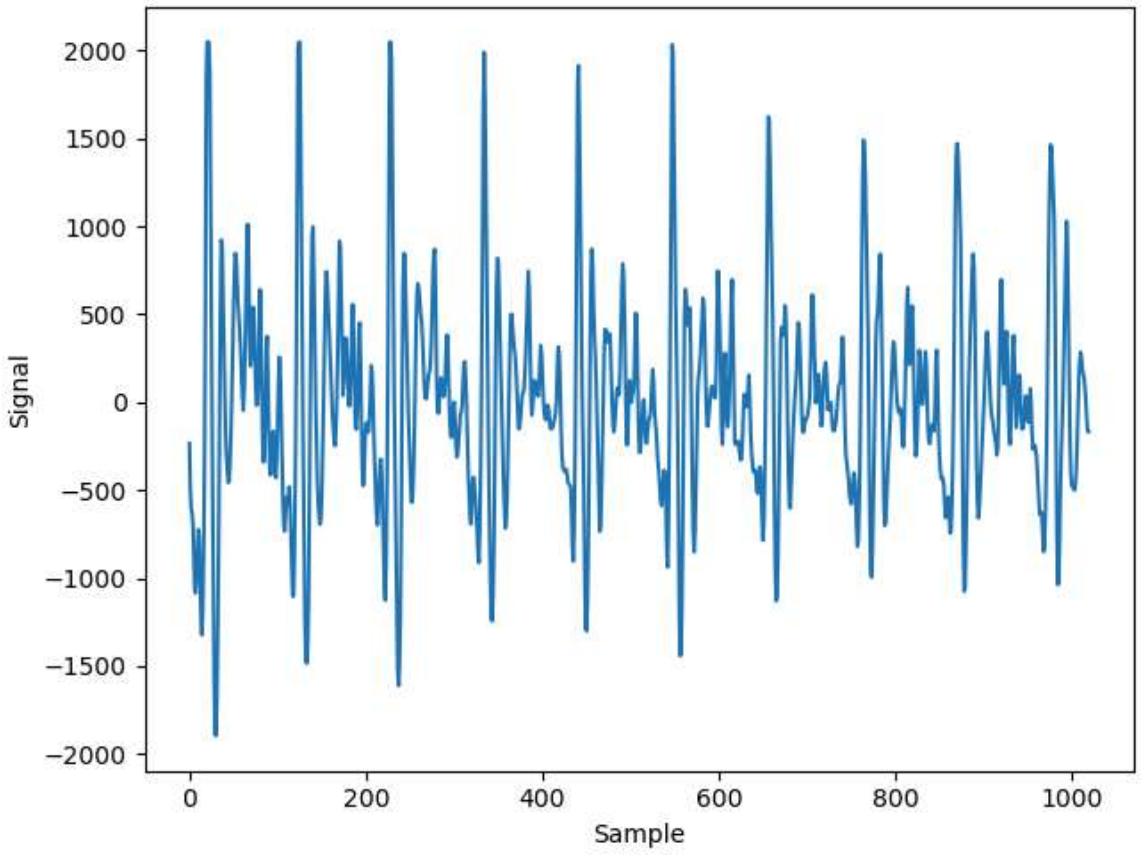
The closer the poles of a signal lie to the unit circle, the more organised and periodic the signal is, indicating stable and regular oscillatory behaviour. As the signal becomes less organised, the poles move inwards, resulting in less oscillation over time. In an ideal scenario involving a purely periodic signal, all poles would lie precisely on the unit circle.

Experiment 3: recovering the excitation (whitening filter)

The signal in `speech.dat` corresponds to the spoken sound */a/*, sampled at 8 kHz. Used in language, we expect this signal to be clearly structured. We start by importing and plotting it:

In [34]:
```
with open(fspc, 'r') as f:
    txt = f.readlines()
    speech = np.array([float(s[:-1]) for s in txt])

speech -= np.mean(speech)

fig = plt.figure(constrained_layout=True)
ax = plt.axes()
plt.plot(speech)
plt.xlabel('Sample')
plt.ylabel('Signal')
plt.show()
```

Figure



Using the functions `ar_order` and `yule_walker` introduced before, we can estimate the optimal model order for this signal, and then the corresponsing model parameters:

In [35]:
```
aro1, _, _ = ar_order(speech, 40)
rho1, sgm1 = yule_walker(speech, order=int(aro1), method="mle")
```

The underlying excitation signal

In the framework of AR modeling, the excitation signal driving the observed speech signal can be estimated using a filtering step as follows:

In [36]:
```
exc1 = lfilter(np.concatenate(([1], -rho1), axis=0), [1], speech)
```

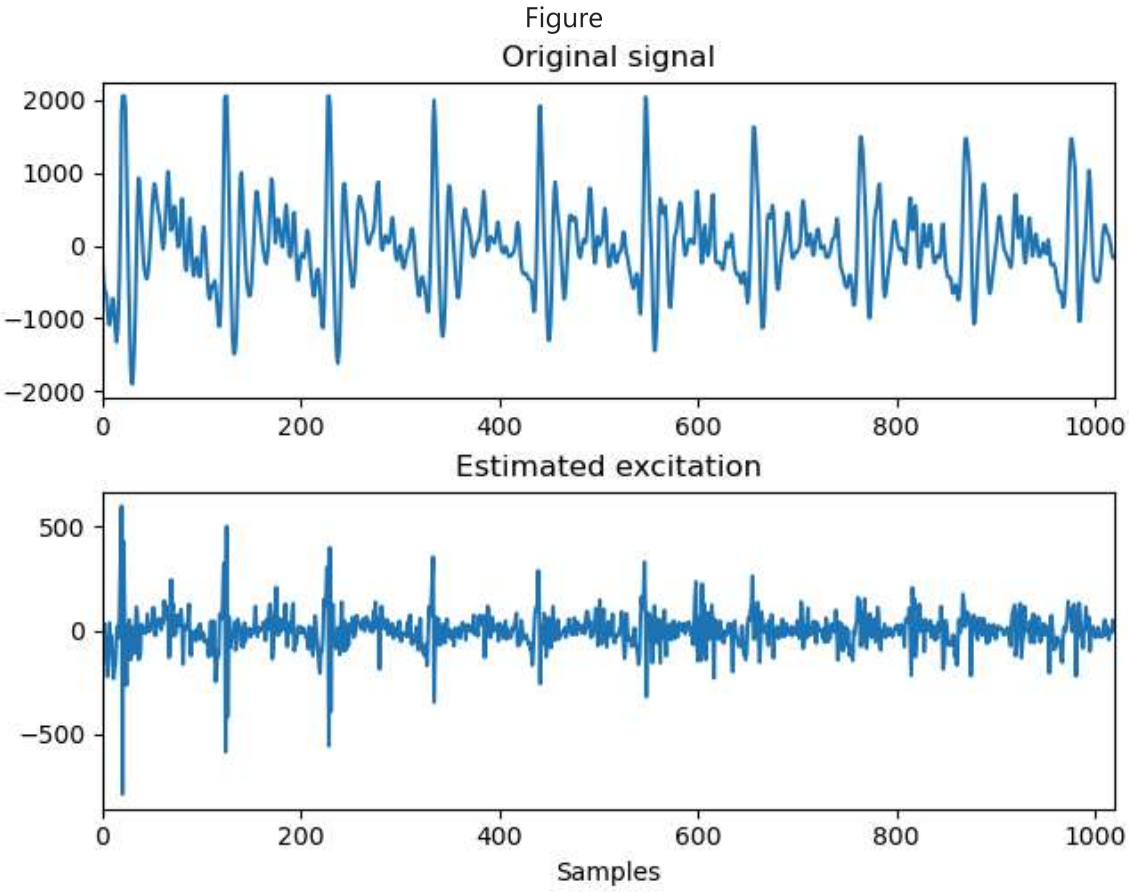
Question 3.1. Explain why the excitation can be estimated in this way.

Answer 3.1. In the AR model, the speech signal is represented as the output of a system whose behavior is described by a set of coefficients. These coefficients capture how each sample of the signal depends on its previous values. Once they are estimated, the inverse of this system can be applied to the signal in order to remove these dependencies. By filtering the speech signal with the estimated AR coefficients, we effectively cancel the predictable structure imposed by the system and obtain the residual part that cannot be predicted from past samples. This residual corresponds to the excitation signal, which is the driving source that originally produced the observed speech.

We can now visualize the estimated excitation signal:

In [ ]:
```
```

In [37]:
```
fig = plt.figure(constrained_layout=True)
plt.subplot(2,1,1)
plt.plot(speech)
plt.title('Original signal')
plt.xlim(0, len(speech))

plt.subplot(2,1,2)
plt.plot(exc1)
plt.title('Estimated excitation')
plt.xlim(0, len(speech))
plt.xlabel('Samples')
plt.show()
```



Question 3.2. Compare the excitation with the speech signal. Does the excitation look like white noise?

Answer 3.2. Visually, the estimated excitation appears much less structured and more random than the original speech signal. The clear periodic patterns present in the speech waveform have disappeared, and the signal now fluctuates rapidly around zero with no evident formant-like structure. From this visual inspection, the excitation seems to behave similarly to white noise, although it is not perfectly white. For voiced sounds like /a/, the excitation may still contain some weak periodic components related to the vibration of the vocal cords, but overall it exhibits the unpredictable, uncorrelated nature characteristic of a whitened signal.

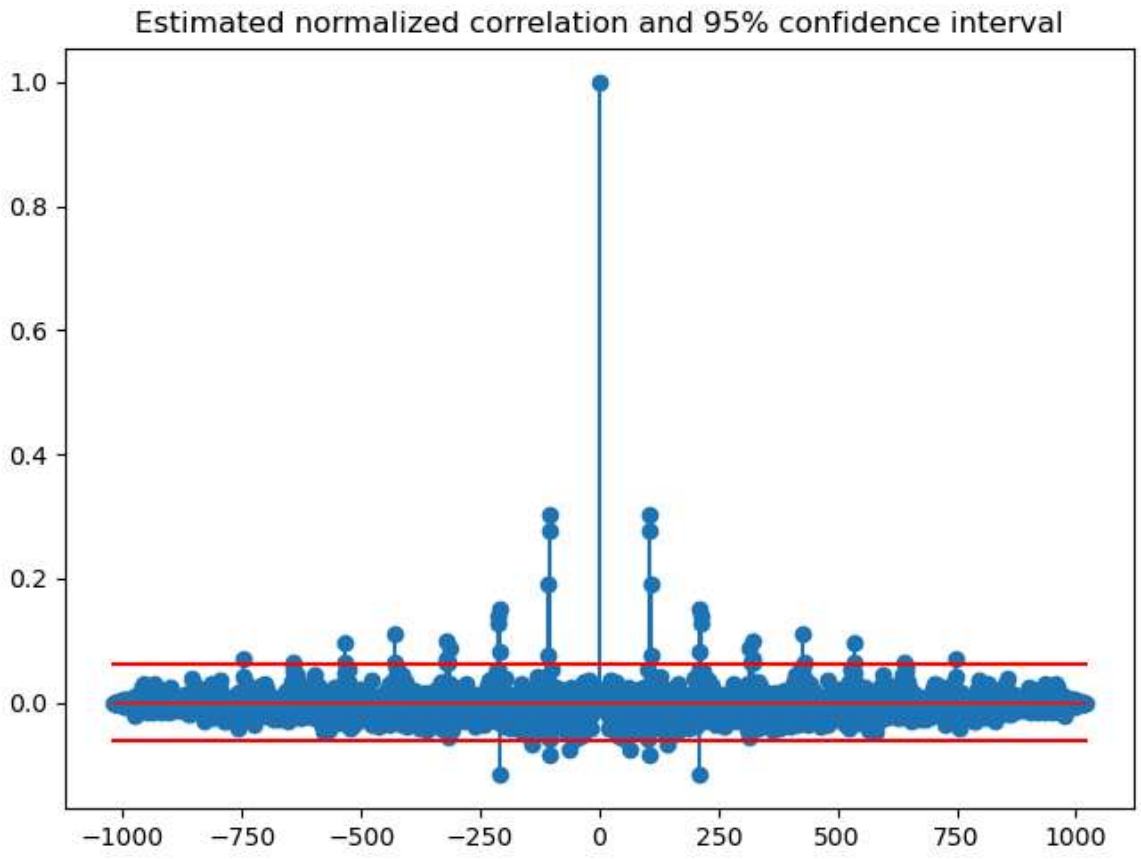
We can test more objectively whether the excitation is indeed similar to white noise by looking at its normalized autocorrelation, as follows:

In [38]:
```
def test_white(x, Aff=0):
    """
    Computation of the ratio of normalized autocorrelation estimates
    larger than a 5% threshold
    x: signal
    Aff: 0 no graphic display; 1 display
    """

    K = len(x)

    # Calculate the biased autocorrelation of the signal
    v = np.correlate(x, x, mode='full') / K
    # Note: K-1 is the index for zero lag
```

```
    thresh = 1.96 / np.sqrt(K)
    pc = np.sum(np.abs(v[K:] / v[K-1]) > thresh) / (K-1)

    if Aff == 1:
        fig = plt.figure(constrained_layout=True)
        ml = K-1 #min(30, K-1)
        lags = range(-ml, ml + 1)
        corr = v[K-1 - ml : K-1 + ml + 1] / v[K-1]
        plt.stem(lags, corr)
        plt.plot([-ml, ml], [+thresh, +thresh], 'r')
        plt.plot([-ml, ml], [-thresh, -thresh], 'r')
        plt.title('Estimated normalized correlation and 95% confidence interval'
        plt.show()

    return pc

print("Proportion above 5% threshold: {:f}".format(test_white(exc1, Aff=1)))
```

```
C:\Users\alexb\AppData\Local\Temp\ipykernel_26136\2118620198.py:19: RuntimeWarnin
g: More than 20 figures have been opened. Figures created through the pyplot inte
rface (`matplotlib.pyplot.figure`) are retained until explicitly closed and may c
onsume too much memory. (To control this warning, see the rcParam `figure.max_ope
n_warning`). Consider using `matplotlib.pyplot.close()`.
  fig = plt.figure(constrained_layout=True)
```

Figure




```
Proportion above 5% threshold: 0.023553
```

Question 3.3. What can we say based on this result?

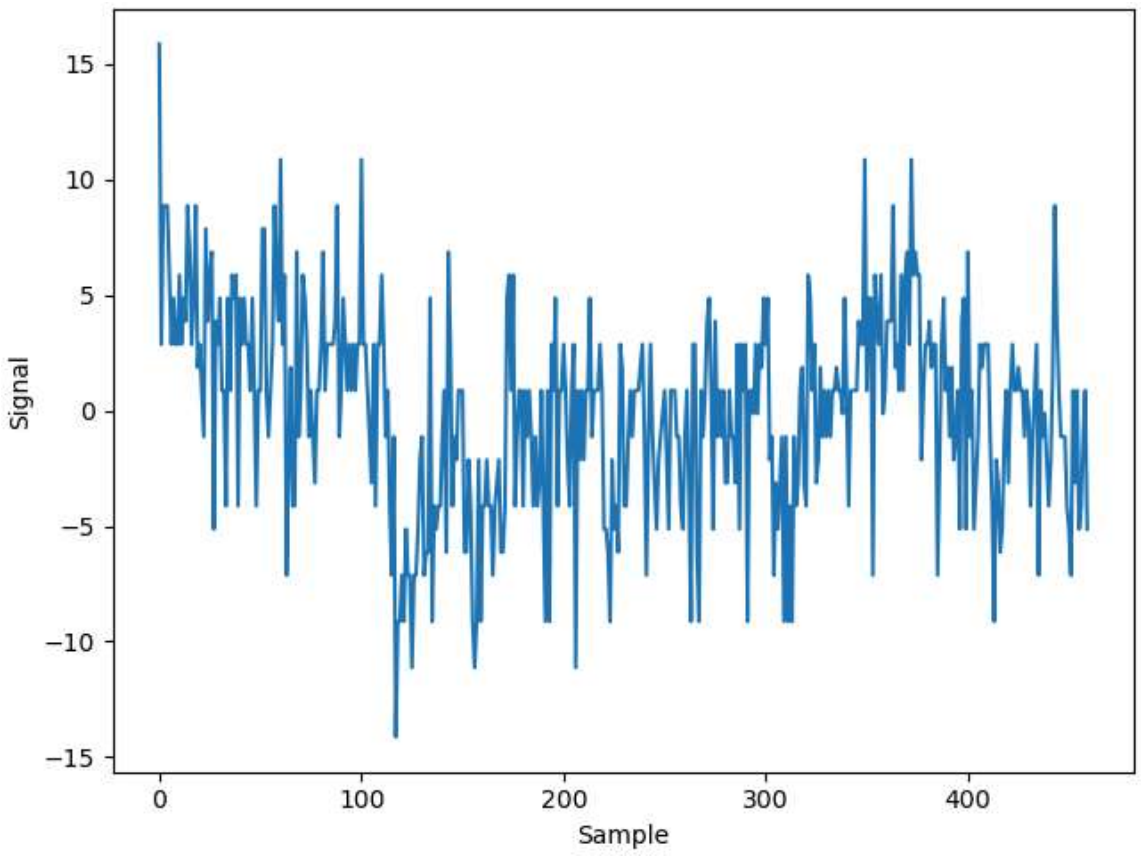
Answer 3.3. The proportion of correlation values exceeding the 5% confidence threshold is approximately 0.02, which is below the expected limit of 0.05 for white noise. This means that only a small fraction of the autocorrelation coefficients lies outside the

confidence interval. Therefore, the excitation can be considered approximately white. In other words, the inverse filtering successfully removed most of the correlations present in the original speech signal, leaving a residual signal that behaves almost like white noise.

We now consider a different example: a timeseries of daily blood systolic pressure recorded from a patient, stored in `blood.dat`:

In [39]:
```
with open(fbld, 'r') as f:
    txt = f.readlines()
    txt = [s[:-1].split() for s in txt]
    blood = np.array([float(s[0]) for s in txt])

blood -= np.mean(blood)

fig = plt.figure(constrained_layout=True)
ax = plt.axes()
plt.plot(blood)
plt.xlabel('Sample')
plt.ylabel('Signal')
plt.show()
```

Figure

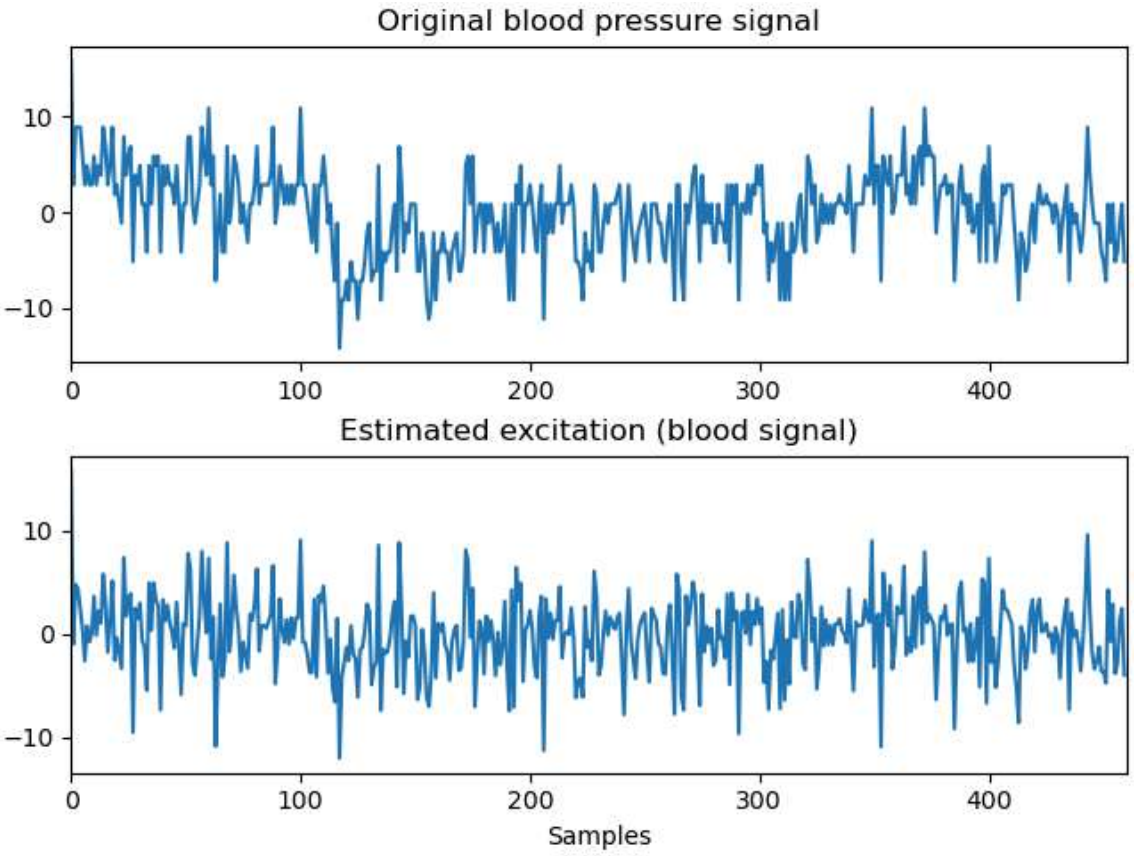


Question 3.4. Repeat the full analysis done for the speech signal (i.e. estimating AR order, model parameters, excitation signal, whiteness test). How does this excitation signal compare to that of the speech example?

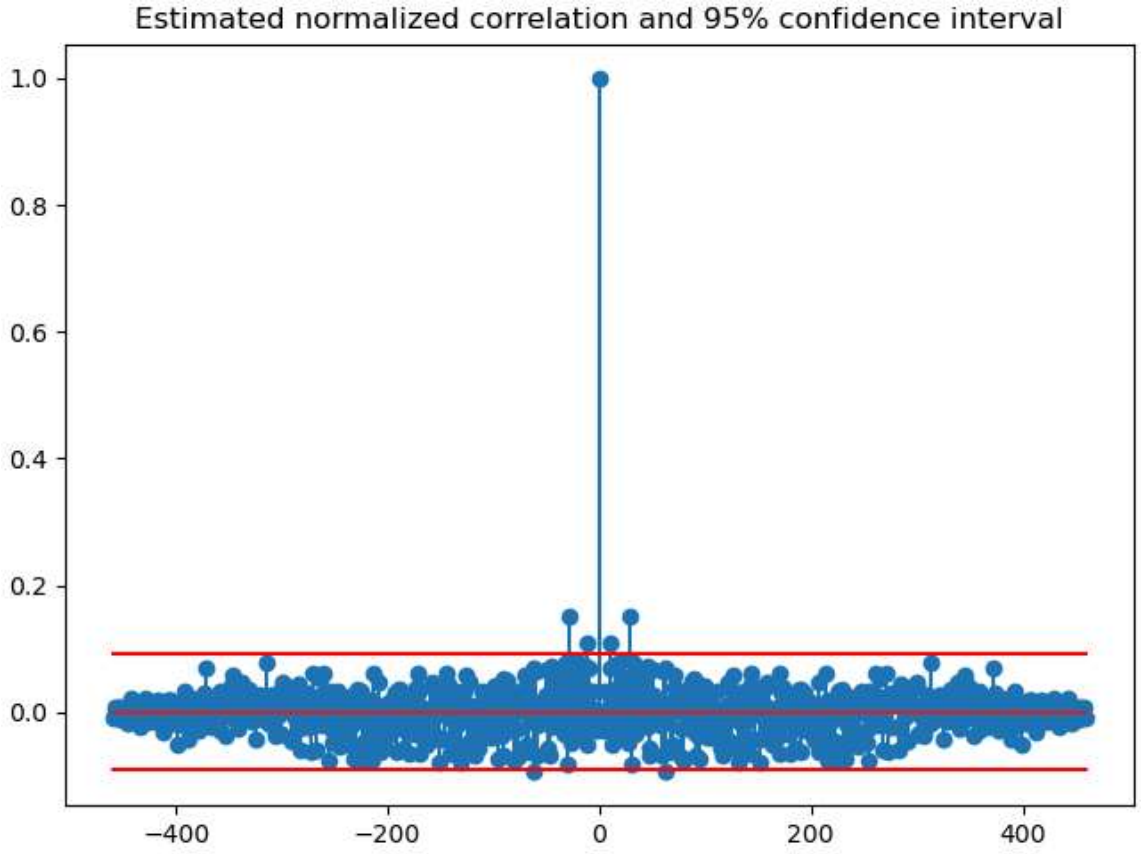
In [40]:
```
#**Answer 3.4.**
# Step 1: Stima ordine AR
ar_blood, _, _ = ar_order(blood, 40)
```

```python
# Step 2: Stima coefficienti AR (Yule-Walker)
rho_blood, sgm_blood = yule_walker(blood, order=int(ar_blood), method="mle")

# Step 3: Calcolo del segnale di eccitazione (whitening filter)
exc_blood = lfilter(np.concatenate(([1], -rho_blood), axis=0), [1], blood)

# Step 4: Plot confronto segnale originale / eccitazione
fig = plt.figure(constrained_layout=True)
plt.subplot(2,1,1)
plt.plot(blood)
plt.title('Original blood pressure signal')
plt.xlim(0, len(blood))

plt.subplot(2,1,2)
plt.plot(exc_blood)
plt.title('Estimated excitation (blood signal)')
plt.xlim(0, len(blood))
plt.xlabel('Samples')
plt.show()

# Step 5: Test di whiteness
prop_blood = test_white(exc_blood, Aff=1)
print(f"Proportion above 5% threshold: {prop_blood:.4f}")
```

Figure



Figure




```
Proportion above 5% threshold: 0.0065
```

Answer 3.4. COMMENTS

(1) Signal inspection and AR modeling

The original DAILY blood pressure signal displays slow oscillations and local trends rather than fast, highly structured variations like the speech waveform. When the AR model is fitted, the estimated order is relatively low, suggesting that only a few past samples contribute significantly to the prediction of the next one. This reflects the smoother, more slowly varying nature of physiological data compared to acoustic signals.

(2) Estimated excitation signal

After applying the whitening filter, the resulting excitation signal appears more irregular and centered around zero. The fluctuations lack any obvious long-term pattern, indicating that the AR filtering has removed some predictable components.

(3) Whiteness test results

The objective whiteness test yields a proportion above 5% threshold of 0.0065, which is very low—well below the 0.05 expected for purely white noise. The normalized autocorrelation plot confirms this: except for the peak at zero lag, nearly all correlation values lie comfortably within the ±95% confidence bounds (the red lines). This means that, from a statistical standpoint, the residual excitation behaves as an approximately white process.

(4) Comparison with the speech example

The AR model successfully removes most correlations from the daily blood pressure series, producing an excitation that passes the whiteness test. However, unlike the speech example — where the excitation represents a real, periodic physiological source — the blood signal's excitation is merely a statistical residual from a slow, non-periodic process. The two cases differ not in model accuracy, but in the underlying timescale and nature of the data: the speech is fast and resonant, the blood pressure is slow and trend-driven. The speech signal is sampled at 8 kHz and contains rich, short-term periodicity due to vocal fold vibrations and resonances of the vocal tract. The blood pressure data, on the other hand, consists of daily samples and represents a slowly varying physiological trend, with no fast periodic driver. Thus, while both residuals can appear "white" in a statistical sense, they describe very different physical realities: Speech excitation: quasi-periodic physical process (vocal folds) → whitening isolates the true excitation source. Blood excitation: slow stochastic process (daily physiological variation) → whitening only removes long-term correlations, not a physical oscillation.

In [ ]: